

Process Concept

- A Process is an instance of a running program.
- A process uses various system resources like memory devices etc.
- Each Linux process has a parent and one or more children.
- The init process is the first user process which is created when the system boots for the first time that continues running until the system is shut down.
- It is the parent process for all the user processes.
- Each and every process has unique non-negative identifier called the process ID.

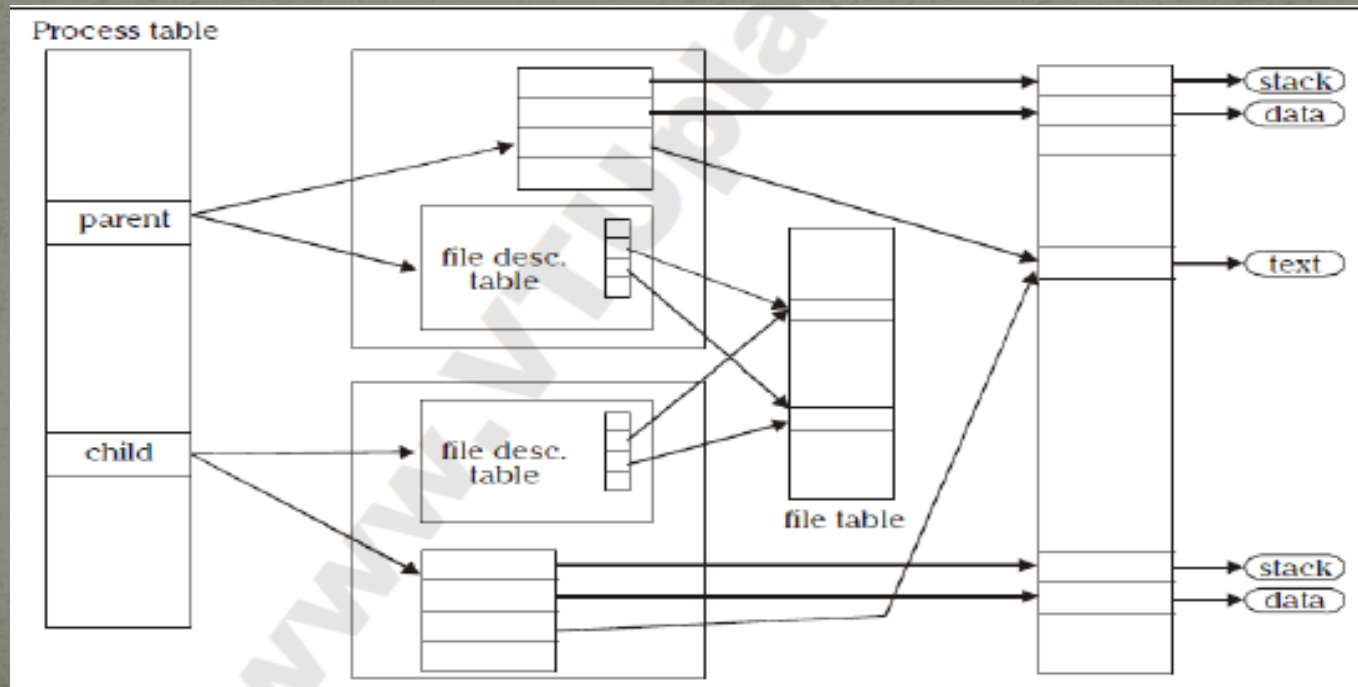
- A process is created by calling the `fork()` or `vfork()` system call.
- A process may execute one or more programs.
- Life time of a process begins when it is created and ends when the process terminates by calling `exit` system call
- When the process having one or more children to terminate, all its processes becomes orphans and are inherited by `init` process
- Linux is a multiprocessing operating system, its objective is to have a process running on each CPU in the system at all times, to maximize CPU utilization.

Kernel Support for Process

- A segment is an area of memory that is managed by the system as a unit.
- A text segment consists of the program text in machine executable instruction code format.
- The data segment contains static and global variables and their corresponding data.
- A stack segment contains runtime variables and the return addresses of all active functions for a process.
- UNIX kernel has a process table that keeps track of all active process present in the system
- Some of these processes belongs to the kernel and are called as “system process”.
- Every entry in the process table contains pointers to the text, data and the stack segments and also to U-area of a process.
- U-area of a process is an extension of the process table entry and contains other process specific data such as the file descriptor table, current root and working directory inode numbers and set of system imposed process limits.

Kernel Support for Process

- All processes in UNIX system expect the process that is created by the system boot code, are created by the fork system call.
- After the fork system call, once the child process is created, both the parent and child processes resumes execution.
- When a process is created by fork, it contains duplicated copies of the text, data and stack segments of its parent as shown in the Figure below.
- Also it has a file descriptor table, which contains reference to the same opened files as the parent, such that they both share the same file pointer to each opened files.



Kernel Support for Process

- Firstly, the existence of a process is represented by a process structure which shows the current status of a process.
- The structure contains the necessary information about a process that is required by the Linux kernel for controlling or scheduling purposes.
- The entries in the process structure include process identifier, parents identifier, process state, address of the loaded program in memory etc.
- A table of process structure is called the process table .
- Initially the table entries are empty . For each process created in the system, an entry is used to keep the process details.
- When a process terminates, its corresponding entry is cleared.

Kernel Support for Process

- Secondly, the user structure is maintained for each process to keep the execution environment that is created for the process.
- This structure contains details such as effective, real user and group identifiers, a pointer to its current working directory, a pointer to its current root directory, a table of pointers to open-files, a signal table showing how this process would respond to each recognized signal in the system, a file creation mask that is used as a default file protection code by the file creation system call, entries to keep the arguments, returned values and error codes of system calls.
- The user structure which is created for the user's login shell is inherited by every created process for that user.

Kernel Support for Process

- Further more , the process is assigned the following attributes which are either inherited from its parent or set by the kernel,

1) A Real User Identification Number (rUID)

The user ID of a user who created the parent process. This is used by the kernel to keep track of who creates which processes on a system.

2) A Real Group Identification Number (rGID)

- The group ID of a user who created the parent process.
- This is used by the kernel to keep track of which group creates which processes on a system.

3) An effective User Identification Number (eUID)

- This is normally the same as the real UID, except when the file that was executed to create the process has its set-UID flag turned on (via the chmod command or API).
- In the case, the process eUID will take on the UID of the file.
- This allows the process to access and create files with the same privileges as the program file owner.

4) An effective Group Identification Number (eGID)

- Same as the real GID except when the file that was executed to create the process has its set-GID flag turned on (via the chmod command or API).
- In that case, the process eGID will take on the GID of the file. This allows the process to access and create files with same privileges as the group to which the program file belongs.

Kernel Support for Process

5) Saved set-UID and Saved set-GID

These are the assigned eUID and eGID, respectively, of the process.

6) Process Group Identification Number (PGID) and selection identification number (SID)

These identify the process group and session of which the process is member.

7) Supplementary Group Identification Numbers

This is a set of additional group IDs for a user who created the process.

8) Current Directory

This is the reference (inode number) to a working directory file.

9) Root Directory

This is the reference (inode number) to a root directory file.

10) Signal Handling

The signal handling settings.

11) Signal Mask

A signal Mask that specifies which signals are to be blocked.

Kernel Support for Process

12).Umask

A file mode mask that is used in creation of files to specify which accession rights should be taken out.

13) Nice Value

The process scheduling priority value.

14) Controlling Terminal

The controlling terminal of the process.

In addition to the above below are the attributes that are different between the parent and child processes,

1) Process Identification Number (PID)

- Integer identification number unique per process in an entire OS.

2) Parent Process Identification Number (PPID)

- The parent process PID

3) Pending signals

- The set of signals that are pending delivery to the parent process.

4) Alarm Clock Time

- The process alarm clock time is reset to zero in the child process

5) File Locks

- Set of file locks owned by the parent process is not inherited by the child process.

PROCESS ATTRIBUTES

- Every process has unique process ID and it is a positive integer.
- In Linux there are some special processes, Process ID '0' is usually the scheduler process (or swapper). It is part of the kernel and is known as a system process.
- Process ID '1' is usually the init process and is invoked by the kernel at the end of the bootstrap procedure. The program file for this process is /etc/init in older versions of Linux and is /sbin/init in newer versions.
- This process is responsible for bringing up a Linux system.
- Init process usually reads the system dependent initialization files (/etc/rc*) and brings the system to a certain state (such as multi user, single user, single user command mode,, etc.,).
- The init process never dies. It is a normal user process not a system process within the kernel.
- Some virtual memory implementations of Linux operating system, process ID 2 is responsible for supporting the paging of the virtual memory system.

PROCESS ATTRIBUTES

- In addition to the process ID, there are other identifiers for every process.
- The following functions return these identifiers. These functions does not have error return values like other functions.

- `#include<sys/types.h>`

- `#include<unistd.h>`

`pid_t getpid (void) :-` returns process ID of calling process

`pid_t getppid (void) :-` returns parent process ID of calling process

`uid_t getuid (void) :-` returns real user ID of calling process

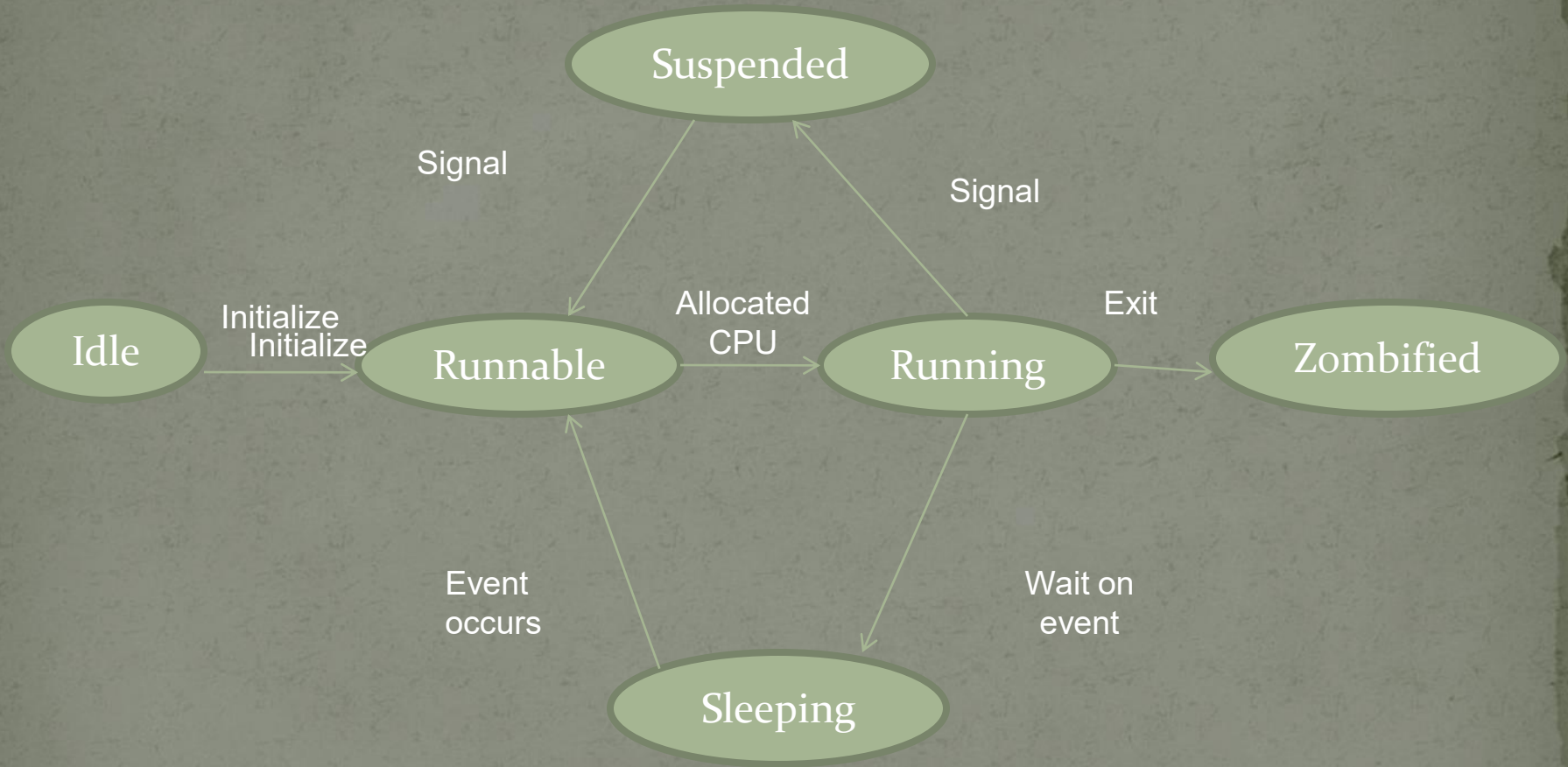
`uid_t geteuid (void) :-` returns effective user ID of calling process

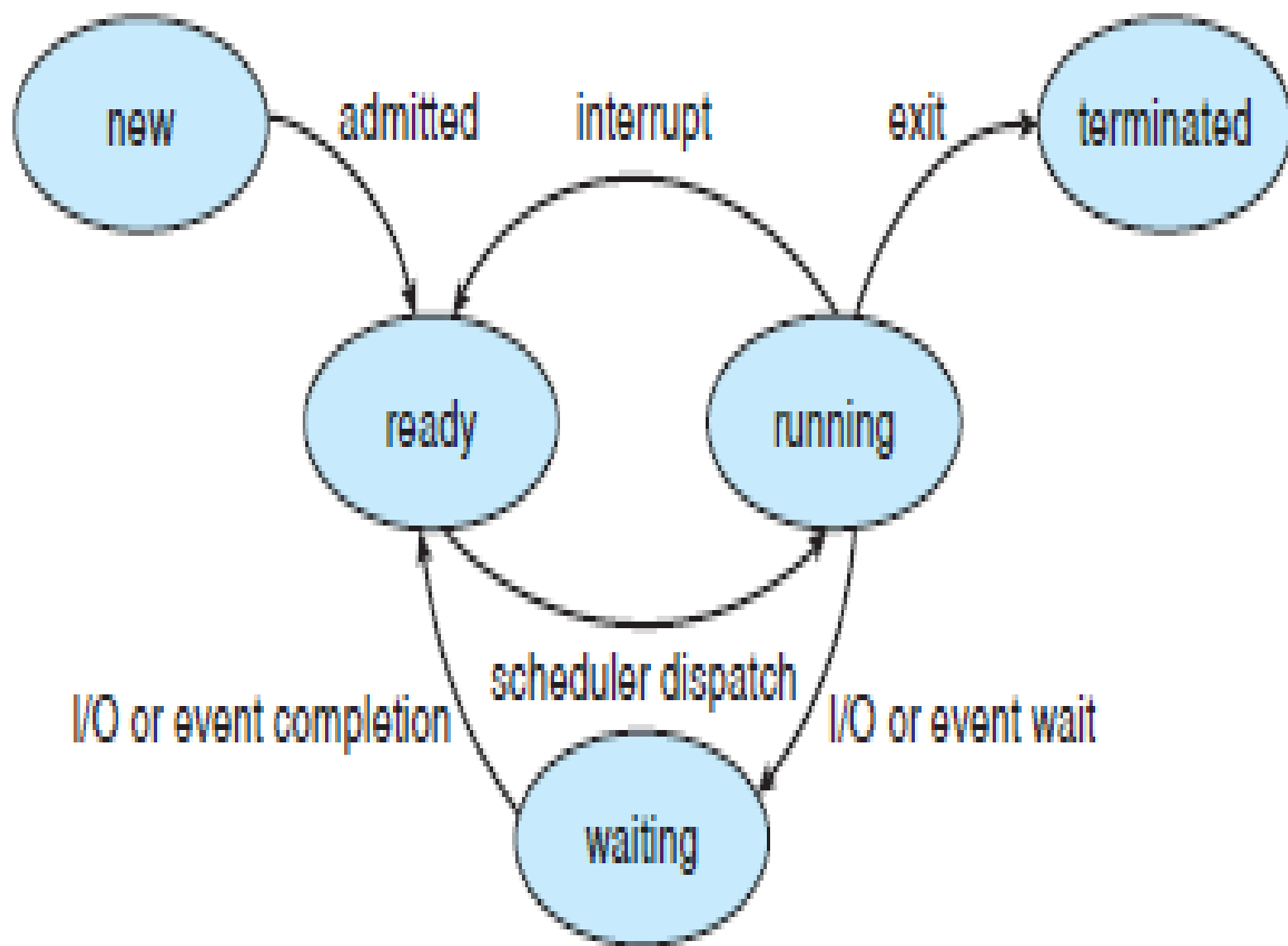
`gid_t getgid (void) :-` returns real group ID of calling process

`gid_t getegid (void) :-` returns effective group ID of calling process

PROCESS STATES

Every process in the system can be in one of six states,





PROCESS STATES

1. Running

Which means that the process is currently using the CPU.

2. Runnable

Process can make use of the CPU as soon as it becomes available.

3. Sleeping

Process is waiting for an event to occur. For example, if a process executes a `sleep()` system call, it sleeps until the I/O request is completed

4. Suspended

The process has been “frozen” by a signal such as `SIGSTOP`. It will resume only when sent a `SIGCONT` signal. For e.g. a `Ctrl+Z` from keyboard suspends all of the processes in the foreground

6. Terminated

process has terminated, but has not yet returned its exit code to its parent. A process remains a zombie until its parent accepts its return code via the `wait()` system call

PROCESS CONTROL

The concept of process control deals with three types of independent mechanisms namely,

- 1) Process Creation
- 2) Process Execution
- 3) Process Termination

1) Process Creation

- In Linux environment a process is created by calling a fork function by any of the existing process created is called a “child process” and the process which has created the new process is called as “parent process” the new process created attains a value ‘0’ where as the parent process is process ID of the new child under the fork function.

Syntax :

```
#include<sys/types.h>
#include<unistd.h>
pid_t fork(void);
```

On creation of the above function creates child process where child process ID is returns to parent and child remains with a value ‘0’ or returns a value ‘-1’ to indicate error.

PROCESS CONTROL

2) Process Execution

- Once the child and parent processes are created, they both start executing the statement.
- During this execution the child process gains few features of the parents like data space, root directory, session ID, process group ID etc.

3) Process Termination

- A process is an instance of an executing program. A process performs certain tasks and terminates. The end of the process is called process termination.
- A process can terminate normally or abnormally.
 1. **Normal Termination of a Process**

A process is said to be terminated normally if,

 - It executes a return statement in the function. This is similar to calling the exit function.
 - It calls the exit functions. This function flushes the buffer and then returns to the kernel.
 - It calls the exit function. This function immediately returns to the kernel without flushing buffers. It is called by the exit function internally.
 2. **Abnormal Termination of a process**

A process is said to be terminated abnormally if,

 - It calls abort function which generates the SIGABRT signal.
 - It receives certain signals that force it to termination

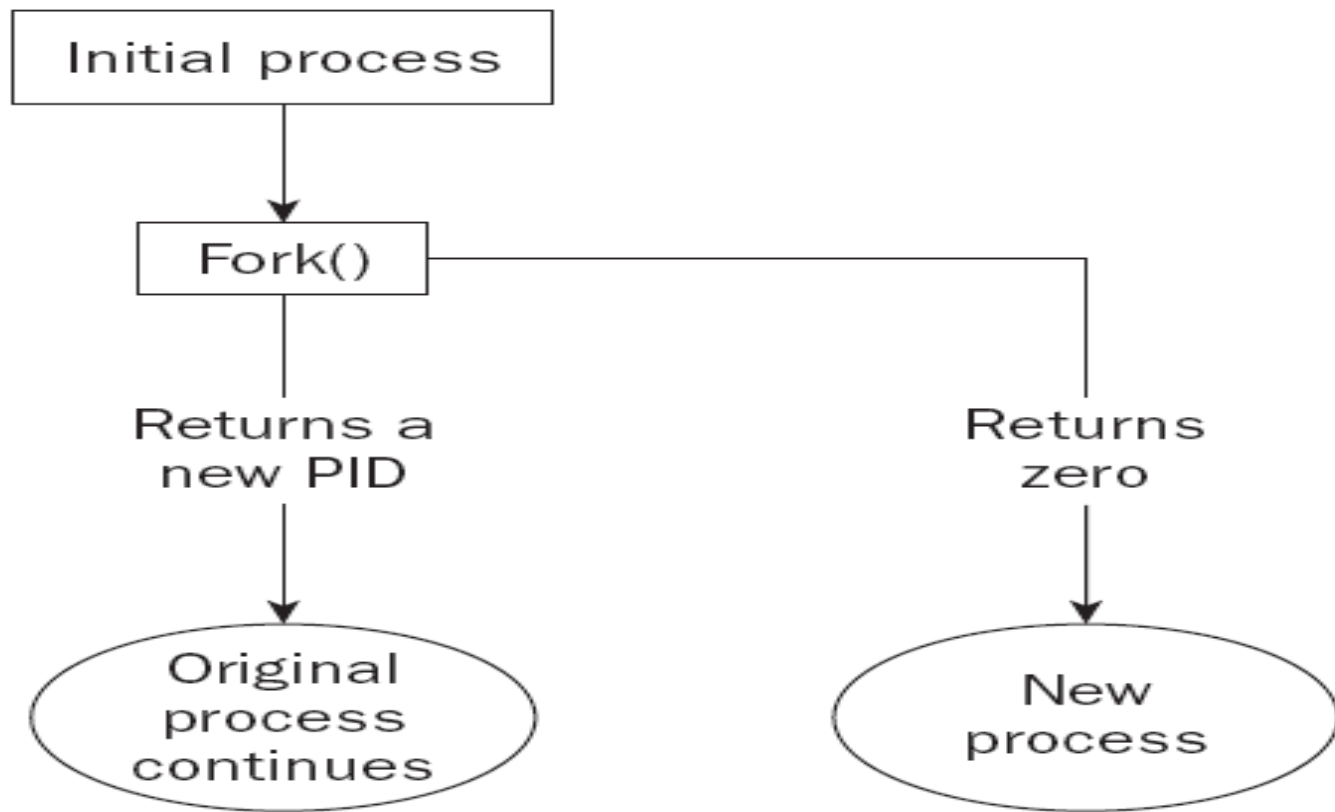
PROCESS CREATION

- The fork() function is used to create a new process. Each time a new process is created, when ever existing process calls a fork() function.
- The fork() function is said to be a “called once and returns twice” function.

Syntax:

```
#include<sys/types.h>  
#include<unistd.h>  
pid_t fork(void);
```

The only difference in the returns is that the return value in the child is 0, whereas the return value in the parent is the process ID of the new child.



Ex:

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    char *message;
    int n;

    printf("fork program starting\n");
    pid = fork();
    switch(pid)
    {
        case -1:
            perror("fork failed");
            exit(1);
        case 0:
            message = "This is the child";
            n = 5;
            break;
        default:
            message = "This is the parent";
            n = 3;
            break;
    }

    for(; n > 0; n--) {
        puts(message);
        sleep(1);
    }
    exit(0);
}
```

- This program runs as two processes. A child is created and prints a message five times.
- The original process (the parent) prints a message only three times. The parent process finishes before the child has printed all of its messages, so the next shell prompt appears mixed in with the output.
- `$ ./fork1`
- fork program starting
- This is the child
- This is the parent
- This is the parent
- This is the child
- This is the parent
- This is the child
- `$ This is the child`
- This is the child

WAITING FOR A PROCESS

- The wait function is used by the parent process to wait for one of its children to terminate and to fetch the child's termination status.

Syntax:

```
#include<sys/types.h>  
#include<sys/wait.h>  
pid_wait(int *status_loc);
```

When a process calls wait then wait,

- 1) Suspends the caller until one of its children terminates.
- 2) Returns immediately with the termination status of a child that terminated.
- 3) Returns immediately with an error if the calling process has no child(either dead or alive).

A successful call to wait returns,

- 1) The process ID of the terminated child.
- 2) Exit status code of the terminated child into the argument status_loc
- 3) Frees proc structure of the terminated child.

PROCESS TERMINATIONS

- A process performs certain tasks and terminates. The end of the process is called process termination. A process can terminate normally or abnormally.

1. Normal Termination of a process

A process is said to be terminated normally if,

- It executes a return statement in the function. This is similar to calling the exit function.
- It calls the exit functions. This function flushes the buffer and then returns to the kernel.
- It calls the exit function. This function immediately returns to the kernel without flushing buffers. It is called by the exit function internally.

2. Abnormal Termination of a process

A process is said to be terminated abnormally if, It calls abort function which generates the SIGABRT signal.

It receives certain signals that force it to termination

- When a process terminates it informs about its termination to its parent process.
- A parent process obtain the child's termination status by calling either the wait or waitpid functions which accepts the status of terminating process as one of the arguments.
- The parent can also examine the termination status of a child by using the macros defined in <sys/wait.h> header.

PROCESS TERMINATIONS

- Macros to Examine the Termination Status

1. WIFEXITED

This macro returns true if the child terminated normally and exit status for a child was returned. To get the low-order 8 bits of the exit status set by the child to `exit` or `_exit` we execute `WEXITSTATUS`.

2. WIFSIGNALED

This macro return true, if the child terminated abnormally and the exit status for a child was returned. To get the signal number that caused the process to terminate we execute `WTERMSIG (status)`.

we can also use the macro `WCOREDUMP (status)` to know whether a core file of the terminated process was generated or not. This macro returns true if file was generated.

3. WIFSTOPPED

This macro returns true if the child process stopped and the exit status for currently stopped process was returned. We execute the macro `WSTOPSIG (status)` to get the signal number that caused the child process to stop.

ZOMBIE PROCESS

What is a zombie process?

- When a process finishes execution, it will have an exit status to report to its parent process. Because of this last little bit of information, the process will remain in the operating system's process table as a *zombie process*, indicating that it is not to be scheduled for further execution, but that it cannot be completely removed (and its process ID cannot be reused) until it has been determined that the exit status is no longer needed.
- When a child exits, the parent process will receive a SIGCHLD signal to indicate that one of its children has finished executing; the parent process will typically call the wait() system call at this point. That call will provide the parent with the child's exit status, and will cause the child to be *reaped*, or removed from the process table.

ZOMBIE PROCESS

How do I see if there are zombie processes on a system?

- Run “ps aux” or ps(1) and look for a Z in the STAT column.

How do I remove zombie processes from a system?

- Well, first you can wait. It’s possible that the parent process is intentionally leaving the process in a zombie state to ensure that future children that it may create will not receive the same pid. Or perhaps the parent is occupied, and will reap the child process momentarily.
- Secondly, you can send a SIGCHLD signal to the parent (“kill -s SIGCHLD <ppid>”). This will cause well-behaving parents to reap their zombie children.
- Finally, you can kill the parent process of the zombie. At that point, all of the parent’s children will be adopted by the init process (pid 1), which periodically runs wait() to reap any zombie children.

```
switch(pid)
{
case -1:
    perror("fork failed");
    exit(1);
case 0:
    message = "This is the child";
    n = 3;
    break;
default:
    message = "This is the parent";
    n = 5;
    break;
}
```

❑ If you run the preceding program with `./fork2 &` and then call the `ps` program after the child has finished but before the parent has finished, you'll see a line such as this. (Some systems may say `<zombie>` rather than `<defunct>`.)

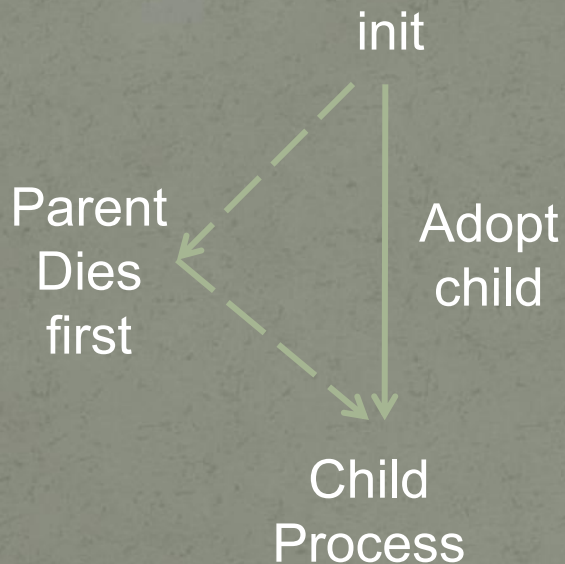

```
$ ps -al
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
004	S	0	1273	1259	0	75	0	-	589	wait4	pts/2	00:00:00	su
000	S	0	1274	1273	0	75	0	-	731	schedu	pts/2	00:00:00	bash
000	S	500	1463	1262	0	75	0	-	788	schedu	pts/1	00:00:00	oclock
000	S	500	1465	1262	0	75	0	-	2569	schedu	pts/1	00:00:01	emacs
000	S	500	1603	1262	0	75	0	-	313	schedu	pts/1	00:00:00	fork2
003	Z	500	1604	1603	0	75	0	-	0	do_exi	pts/1	00:00:00	fork2 <defunct>
000	R	500	1605	1262	0	81	0	-	781	-	pts/1	00:00:00	ps

- If the parent then terminates abnormally, the child process automatically gets the process with PID 1 (init) as parent.
- The child process is now a zombie that is no longer running but has been inherited by init because of the abnormal termination of the parent process.
- The zombie will remain in the process table until collected by the init process. The bigger the table, the slower this procedure. You need to avoid zombie processes, because they consume resources until init cleans them up

ORPHAN PROCESS

- If a parent dies before its child, the child is automatically adopted by the original “init” process, PID 1.



**PROCESS
ADOPTION**

PROCESS APIs

- A system call is a request for the operating system to do something on behalf of the user's program.
- System calls are functions used in the kernel itself.

Fork Function:

- An existing process can create a new one by calling the fork function.

```
#include <unistd.h>
```

```
pid_t fork(void);
```

Returns: 0 in child, process ID of child in parent, -1 on error.

The new process created by fork is called the child process.

This function is called once but returns twice. The only difference in the returns is that the return value in the child is 0, whereas the return value in the parent is the process ID of the new child.

PROCESS APIs

vfork :

- The modified version of `fork()` is `vfork()`
- The function `vfork` has the same calling sequence and same return values as `fork`.
- The `vfork` function creates the new process, just like `fork`, without copying the address space of the parent into the child, as the child won't reference that address space.
- Instead, while the child is running and until it calls either `exec` or `exit`, the child runs in the address space of the parent. This optimization provides an efficiency gain on some paged virtual-memory implementations of the UNIX System.
- `vfork` guarantees that the child runs first, until the child calls `exec` or `exit`. When the child calls either of these functions, the parent resumes.

Basis for Comparison

fork()

vfork()

Basic

Child process and parent process has separate address spaces.

Child process and parent process shares the same address space.

Execution

Parent and child process execute simultaneously.

Parent process remains suspended till child process completes its execution.

Modification

If the child process alters any page in the address space, it is invisible to the parent process as the address space are separate.

If child process alters any page in the address space, it is visible to the parent process as they share the same address space.

Copy-on-write

fork() uses copy-on-write as an alternative where the parent and child shares same pages until any one of them modifies the shared page.

vfork() does not use copy-on-write.

PROCESS APIs

exit :

A process can terminate normally in five ways:

1. Executing a return from the main function.
2. Calling the exit function.
3. Calling the _exit or _Exit function.
4. Executing a return from the start routine of the last thread in the process.
5. Calling the pthread_exit function from the last thread in the process.

Once the process terminates, the kernel closes all the open descriptors for the process, releases the memory that it was using.

Three functions terminate a program normally: _exit and _Exit, which return to the kernel immediately, and exit, which performs certain cleanup processing and then returns to the kernel.

```
#include <stdlib.h>
```

```
void exit( int status);
```

```
void _Exit( int status);
```

```
#include <unistd.h>
```

```
void _exit( int status);
```


PROCESS APIs

wait and waitpid Functions:

```
#include <sys/ wait.h>
```

```
pid_t wait(int *statloc);
```

```
pid_t waitpid( pid_t pid, int *statloc, int options);
```

Both return: process ID if OK, or -1 on error.

- The wait function can block the caller until a child process terminates, whereas waitpid has an option that prevents it from blocking.
- The waitpid function doesn't wait for the child that terminates first; it has a number of options that control which process it waits for.

For both functions, the argument statloc is a pointer to an integer. If this argument is not a null pointer, the termination status of the terminated process is stored in the location pointed to by the argument.

The interpretation of the pid argument for waitpid depends on its value:

pid == 1 Waits for any child process.

pid > 0 Waits for the child whose process ID equals pid.

pid == 0 Waits for any child whose process group ID equals that of the calling process.

PROCESS APIs

`pid < 1` Waits for any child whose process group ID equals the absolute value of `pid`.

The options constants for `waitpid`:

`WCONTINUED`

`WNOHANG`

`WUNTRACED`

The `waitpid` function returns the process ID of the child that terminated and stores the child's termination status in the memory location pointed to by `statloc`.

exec Functions:

- Exec replaces the current process its text, data, heap, and stack segments with a brand new program from disk.
- with the exec functions, we can initiate new programs.

There are six different exec functions.

PROCESS APIs

There are six different exec functions.

```
#include <unistd.h>
```

```
int execl (const char *pathname, const char *argo, ... /* (char *)o */ );
```

```
int execv (const char *pathname, char *const argv []);
```

```
int execlp (const char *pathname, const char *argo, ... /* (char *)o, char  
    *const envp[] */ );
```

```
int execve (const char *pathname, char *const argv[], char *const envp []);
```

```
int execlp (const char *filename, const char *argo, ... /* (char *)o */ );
```

```
int execvp (const char *filename, char *const argv []);
```


PROCESS APIs

There are six different exec functions.

```
#include <unistd.h>
```

```
int execl (const char *pathname, const char *argo, ... /* (char *)o */ );
```

```
int execv (const char *pathname, char *const argv []);
```

```
int execl (const char *pathname, const char *argo, ... /* (char *)o, char *const  
envp[] */ );
```

```
int execve (const char *pathname, char *const argv[], char *const envp []);
```

```
int execlp (const char *filename, const char *argo, ... /* (char *)o */ );
```

```
int execvp (const char *filename, char *const argv []);
```

All six return: 1 on error, no return on success.

- The first difference in these functions is that the first four take a pathname argument, whereas the last two take a filename argument. When a filename argument is specified
 - If filename contains a slash, it is taken as a pathname.
 - Otherwise, the executable file is searched for in the directories specified by the PATH environment variable.

PROCESS APIs

- The functions `execl`, `execlp`, and `execle` require each of the command-line arguments to the new program to be specified as separate arguments. We mark the end of the arguments with a null pointer.
- For the other three functions (`execv`, `execvp`, and `execve`), we have to build an array of pointers to the arguments, and the address of this array is the argument to these three functions.

SIGNALS: Introduction

Signals:

- Signals are software interrupts. Signals provide a way of handling asynchronous events.
- Every signal has a name. These names all begin with the three characters SIG. These names are all defined by positive integer constants (the signal number) in the header `<signal.h>`.
- No signal has a signal number of 0.

1) Sending signals using keyboard

- Ctrl-c** : causes the system to send INT signal (SIGINT) to the running process. By default, this signal causes the process to immediately terminate.
- Ctrl-z** : causes the system to send TSTP signal (SIGTSTP) to the running process. By default, the signal causes the process to suspend execution.
- Ctrl-** : causes the system to send ABRT signal (SIGABRT) to the running process. By default this signal causes the process to immediately terminate.

SIGNALS: Introduction

2) Another way of sending signals is by various commands

i) **Kill:** It accepts two parameters, a signal name (or number), and a process ID

Syntax: Kill <signal> <PID>

Ex: kill -int 5342 same as Ctrl-c

ii) **fg:** it will resume execution of the process(that was suspended with Ctrl-Z)

Signal Generation and Handling

POSIX Signals

SIGABRT--Abnormal termination signal caused by the `abort()` function. A portable program should void catching **SIGABRT**.

SIGALRM--The timer set by the `alarm()` function has timed-out

SIGFPE --Arithmetic exception, such as overflow or division by zero.

SIGHUP --Hang-up detected on controlling terminal or death of a controlling process.

SIGILL --Illegal instruction indicating a program error. Applications may wish to catch this signal and attempt to recover from bugs. A portable program should not intentionally generate illegal instructions. After a **SIGILL** is caught, the only portable thing to do is to `siglongjmp()` back to a known place in your program (or `callexit()`).

SIGINT --Interrupt special character typed on controlling keyboard.
SIGKILL Termination signal. This signal cannot be caught or ignored.

Signal Generation and Handling

SIGPIPE --Write to a pipe with no readers.

SIGQUIT --Quit special character typed on controlling keyboard.

SIGSEGV --Invalid memory reference. Like SIGILL, portable programs should not intentionally generate invalid memory references.

SIGTERM --Termination signal.

SIGUSR1 -- Reserved to be defined by user1.

SIGUSR2 -- Reserved to be defined by user2

Job Control Signals (POSIX.1)

SIGCHLD --Child process terminated or stopped. By default, this signal is ignored.

SIGCONT --Continue the process if it is currently stopped; otherwise, ignore the signal.

SIGSTOP --Stop signal. This signal cannot be caught or ignored.

SIGTSTP --Stop special character typed on the controlling keyboard.

SIGTTIN --Read from the controlling terminal attempted by a member of a background process group.

SIGTTOU --Write to controlling terminal attempted by a member of a background process group

Signal Generation and Handling

Simple program to catch SIGUSR1 & SIGUSR2

```
static void sig_usr (int);
int main(void)
{
    if (signal(SIGUSR1, sig_usr) == SIG_ERR) err_sys("can't catch SIGUSR1");
    if (signal(SIGUSR2, sig_usr) == SIG_ERR) err_sys("can't catch SIGUSR2");
    for ( ; ; )
        pause();
}
static void sig_usr (int signo)
{
    if (signo == SIGUSR1)
        printf("received SIGUSR1\n");
    else if (signo == SIGUSR2)
        printf("received SIGUSR2\n");
    else err_dump("received signal %d\n", signo);
}
```

Kernel Supports of Signals

- When a signal is generated for a process, the kernel will set the corresponding signal flag in the process table slot of the recipient process.
- If the recipient process is asleep, the kernel will awaken the process by scheduling it.
- When the recipient process runs, the kernel will check the process U-area that contains an array of signal handling specifications.
- If array entry contains a zero value, the process will accept the default action of the signal.
- If array entry contains a 1 value, the process will ignore the signal and kernel will discard it.
- If array entry contains any other value, it is used as the function pointer for a user -defined signal handler routine

Signal Function

Signal Function:

- The simplest interface to the signal features of the UNIX System is the signal function.

```
#include <signal.h>
```

```
void (*signal(int signo, void (*func)(int)))(int);
```

Returns: previous disposition of signal if OK, SIG_ERR on error.

- The signo argument is just the name of the signal.
- func defines how to handle signals like ignore, default etc
- The value of func is (a) the constant SIG_IGN, (b) the constant SIG_DFL, or (c) the address of a function to be called when the signal occurs.

Unreliable Signals

Unreliable Signals:

Unreliable signals mean that signals could get lost: a signal could occur and the process would never know about it. Also, a process had little control over a signal: a process could catch the signal or ignore it.

Interrupted System Calls:

- It is a system call within the kernel that is interrupted when a signal is caught.
- To support this feature, the system calls are divided into two categories: the "slow" system calls and all the others.
- The slow system calls are those that can block forever.
- Reads that can block the caller forever if data isn't present with certain file types.
- Writes that can block the caller forever if the data can't be accepted immediately by these same file types.
- Opens that block until some condition occurs on certain file types.
- The pause function and the wait function.
- Certain ioctl operations.
- Some of the interprocess communication functions .

kill and raise functions

kill and raise Functions:

- The kill function sends a signal to a process or a group of processes.
- The raise function allows a process to send a signal to itself.

```
#include <signal.h>
```

```
int kill( pid_t pid, int signo);
```

```
int raise(int signo);
```

Both return: 0 if OK, -1 on error.

There are four different conditions for the pid argument to kill.

pid > 0 The signal is sent to the process whose process ID is pid.

pid == 0 The signal is sent to all processes whose process group ID equals the process group ID of the sender and for which the sender has permission to send the signal.

pid < 0 The signal is sent to all processes whose process group ID equals the absolute value of pid and for which the sender has permission to send the signal.

alarm And pause Functions

pid == 1 The signal is sent to all processes on the system for which the sender has permission to send the signal.

- The super user can send a signal to any process.

Alarm function:

- The alarm function allows us to set a timer that will expire at a specified time in the future.
- When the timer expires, the SIGALRM signal is generated.

```
#include <unistd.h>
```

```
unsigned int alarm(unsigned int seconds);
```

Returns: 0 or number of seconds until previously set alarm.

The seconds value is no. of clock seconds in the future when the signal should be generated.

alarm And pause Functions

- If, when we call alarm, a previously registered alarm clock for the process has not yet expired, the number of seconds left for that alarm clock is returned as the value of this function.
- If a previously registered alarm clock for the process has not yet expired and if the seconds value is 0, the previous alarm clock is canceled. The number of seconds left for that previous alarm clock is still returned as the value of the function.

pause function:

- The pause function suspends the calling process until a signal is caught.

#include <unistd.h>

int pause(void);

Returns: 1 with errno set to EINTR

- The only time pause returns is if a signal handler is executed and that handler returns.

abort And sleep Functions

abort Function:

- The abort function causes abnormal program termination.

```
#include <stdlib.h>
```

```
void abort(void);
```

This function never returns.

- This function sends the SIGABRT signal to the caller.

system Function:

- It is convenient to execute a command string from within a program.

```
#include <stdlib.h>
```

```
int system(const char *cmdstring);
```

- If cmdstring is a null pointer, *system* returns nonzero only if a command processor is available. This feature determines whether the *system* function is supported on a given operating system. Under the UNIX System, *system* is always available.

abort And sleep Functions

The *system* is implemented by calling `fork`, `exec`, and `waitpid`, there are three types of return values.

- If either the `fork` fails or `waitpid` returns an error, `system` returns 1 with `errno` set to indicate the error.
- If the `exec` fails, implying that the shell can't be executed, the return value is as if the shell had executed `exit(127)`.
- Otherwise, all three functions `fork`, `exec`, and `waitpid` succeed, and the return value from *system* is the termination status of the shell, in the format specified for `waitpid`.

Sleep Function:

```
#include <unistd.h>
```

```
unsigned int sleep(unsigned int seconds);
```

Returns: 0 or number of unslept seconds.

This function causes the calling process to be suspended until either

- The amount of wall clock time specified by `seconds` has elapsed. The return value is 0.
- A signal is caught by the process and the signal handler returns. Return value is the number of unslept seconds.